AI SOFTWARE ENGINEERING / BLOCCO 3 / LEZIONE 12

Tactical DDD e Consistency Boundaries

Dalle regole del dominio agli Aggregate, alle transazioni affidabili e agli eventi tra contesti.

Dispensa di studio · 35–45 minuti di teoria e lettura, con laboratorio progressivo separato

Prerequisiti: lezioni 1–11 del percorso; Strategic DDD e Context Mapping.

Caso guida: Reporting → Operations → Billing. Tutte le policy indicate come “approvate” sono approvate soltanto per l’esercizio, non per un prodotto reale.

Approccio: framework-independent, con esempi TypeScript, SQL e pseudocodice.

Materiale didattico originale · Fonti tecniche verificate il 2 novembre 2026

Guida allo studio e collegamento con la Lezione 11

La Lezione 11 ha individuato tre Bounded Context: Reporting, Operations e Billing. Ha stabilito il significato dello Strategic DDD: i confini del modello e la relazione tra sistemi sono prima di tutto decisioni di dominio e organizzative. La Lezione 12 entra in un singolo contesto, Operations, e studia come le regole diventano comportamenti eseguibili senza lasciare il loro significato nelle mani dei controller, dei database o degli agenti AI.

L'obiettivo non è trasformare ogni requisito in un'architettura sofisticata. È progettare un modello sufficientemente espressivo da rendere difficile violare le business policy approvate e sufficientemente semplice da poter essere modificato, verificato e mantenuto.

Idea guida: gli Aggregate non si disegnano a partire dalle tabelle. Si disegnano a partire dagli invarianti che devono restare veri quando una modifica viene confermata.

Percorso concettuale

1. Entity e Value Object → 2. Aggregate e Aggregate Root → 3. consistenza transazionale e concorrenza → 4. Domain Services e Domain Events → 5. Eventual Consistency e affidabilità → 6. specifiche, test e verifica indipendente.

Stato delle informazioni

Categoria	Nel caso guida
KNOWN	Reporting, Operations e Billing sono i contesti utilizzati didatticamente nella Lezione 11.
DECIDED FOR LAB	Un WorkOrder si completa soltanto se assegnato e dotato di evidenza con rapporto e durata positiva.
UNKNOWN	Se il completamento autorizza la fatturazione; SLA di propagazione, policy di recupero e versionamento reale.
NON-GOAL	Non implementare una soluzione produttiva, non decretare microservizi obbligatori, non inventare policy Billing.

1. Entity: identità, continuità e comportamento

Una Entity è un concetto del dominio che deve essere riconoscibile nel tempo. Anche quando cambiano gli attributi, la sua identità rimane. Due richieste di manutenzione con stesso indirizzo, descrizione e data non sono necessariamente la stessa richiesta; gli esperti di dominio potrebbero doverle seguire, assegnare e rendicontare separatamente.

Una Entity non è semplicemente una riga SQL e non coincide con la presenza di una colonna ID. La sua identità appartiene al linguaggio del dominio e serve a preservare il significato dei cambiamenti nel tempo.

Quando il dominio contiene regole importanti, una Entity ricca espone metodi che descrivono comportamenti. Una classe con setter pubblici per ogni proprietà trasferisce invece la responsabilità delle verifiche ai consumer, moltiplicando i punti in cui una regola può essere dimenticata.

```
// Approccio povero di semantica
workOrder.status = "COMPLETED";

// Approccio orientato al dominio
workOrder.complete(completionEvidence);
```

Questi metodi non sono garanzia sufficiente da soli: la persistenza, la concorrenza e i vincoli trasversali devono essere coerenti con ciò che il modello promette. Tuttavia costituiscono un punto di ingresso controllato e leggibile anche per un agente AI.

Domanda di Domain Discovery

Se due elementi hanno tutti i valori identici, il business deve poterli distinguere e seguirli indipendentemente? Se sì, un modello Entity è spesso appropriato. Se no, può essere più adatto un Value Object.

2. Value Object: significato dei dati senza identità

Un Value Object viene riconosciuto dall'insieme dei valori che contiene. In un modello idiomatico è immutabile: sostituire un importo significa creare un nuovo importo, non modificare in modo incontrollato l'oggetto esistente.

Un Value Object è particolarmente utile quando un tipo primitivo nasconde un concetto importante. Il numero 15000, senza un tipo, può indicare centesimi, secondi, millimetri o un ID. Un oggetto Money chiarisce valuta, unità e operazioni consentite. Il suo costruttore e i suoi metodi possono preservare invarianti locali.

```
class Money {
  constructor(
    readonly cents: number,
    readonly currency: string
  ) {
    if (!Number.isSafeInteger(cents)) {
      throw new Error("Amount must be an integer");
    }
    if (!/^[A-Z]{3}$/.test(currency)) {
      throw new Error("Invalid currency code");
    }
  }

  add(other: Money): Money {
    if (other.currency !== this.currency) {
      throw new Error("Currency mismatch");
    }
    return new Money(this.cents + other.cents,
                     this.currency);
  }
}
```

Il codice è illustrativo, non pronto per la gestione monetaria produttiva: servono vincoli di overflow, arrotondamenti, rappresentazione delle valute e regole di business esplicite. Il principio resta valido in PHP, Python, Java o TypeScript.

Un indirizzo può essere Value Object nel contesto di consegna, ma diventare Entity se deve conservare una storia autonoma di verifiche e autorizzazioni. La classificazione non è una proprietà universale della parola “indirizzo”: dipende dal Bounded Context.

3. Aggregate: confine di consistenza del dominio

Un Aggregate è una unità logica del modello che contiene una Aggregate Root ed eventualmente altre Entities e Value Objects. Tutte le modifiche coerenti con il confine devono essere governate dalla Root. Il punto importante non è che gli oggetti siano correlati nel database, bensì quali invarianti devono rimanere veri alla conclusione dell’operazione.

Un Aggregate è principalmente un confine di consistenza. L’Aggregate Root è il punto di ingresso che protegge quel confine.

Caso WorkOrder

Nel laboratorio, WorkOrder è l’Aggregate Root. Conserva identità, stato, riferimento al tecnico e, quando completato, CompletionEvidence. Un Technician potrebbe essere un Aggregate autonomo: WorkOrder conserva soltanto il suo identificativo per evitare che il ciclo di vita del tecnico diventi parte della stessa unità transazionale.

Elemento	Scelta didattica	Ragione
WorkOrder	Entity e Root	Ha identità e un ciclo di vita autonomo; protegge il completamento.
CompletionEvidence	Value Object	Rapporto e durata con validazione dei valori; non è richiesta identità autonoma.
TechnicianId	Riferimento esterno	La vita del tecnico non dipende dalla modifica del WorkOrder.
BillingCandidate	Altro contesto	La fatturabilità non appartiene all’autorità di Operations.

Invarianti di laboratorio

R1: completamento permesso soltanto nello stato ASSIGNED. **R2:** completamento con riferimento a rapporto non vuoto e durata di lavoro positiva. Sono due regole decise soltanto per questa esercitazione. Non affermano nulla sulle policy effettive di FacileGestire.

```
class CompletionEvidence {
  private constructor(
    readonly reportRef: string,
    readonly minutesWorked: number
  ) {}

  static create(ref: string, minutes: number) {
    if (!ref.trim()) throw new Error("Report required");
    if (!Number.isSafeInteger(minutes) || minutes <= 0) {
      throw new Error("Invalid duration");
    }
```

```
      return new CompletionEvidence(ref, minutes);
    }
  }

  class WorkOrder {
    constructor(
      readonly id: string,
      private status: "OPEN" | "ASSIGNED" | "COMPLETED",
      private technicianId: string | null
    ) {}

    complete(evidence: CompletionEvidence) {
      if (this.status !== "ASSIGNED" ||
          this.technicianId === null) {
        throw new Error("WorkOrder not completable");
      }
      this.status = "COMPLETED";
      // Collect domain event; persistence follows.
    }
  }
```

In un progetto reale il modello dovrebbe definire anche creazione, assegnazione, annullamento, autorizzazioni, policy di modifica delle evidenze e gestione completa degli errori. Il codice qui non è una libreria di dominio completa: illustra il luogo della decisione.

4. Come individuare un buon confine

Una relazione non determina automaticamente la membership nello stesso Aggregate. WorkOrder ha un immobile, un tecnico, documenti e potenzialmente effetti economici; ciò non giustifica l'inclusione dell'intera anagrafica immobiliare, dei calendari dei tecnici e delle fatture nella stessa Root.

Un Aggregate troppo grande produce operazioni costose, lock e conflitti concorrenti frequenti. Un Aggregate troppo piccolo, se separa una regola che richiede atomicità, espone invece gli invarianti a modifiche parziali. In entrambi i casi il problema nasce dall'assenza di un'analisi delle regole.

Checklist progettuale

- Quali condizioni devono essere sempre vere quando la transazione termina?
- Quali oggetti hanno un proprio ciclo di vita e possono essere salvati indipendentemente?
- Quali effetti possono essere propagati dopo il commit?
- La contesa concorrente è prevista? Quanto costa un conflitto?
- Un vincolo trasversale richiede una capacità di reservation o un diverso confine?

Vaughn Vernon propone, come orientamento iniziale, di modellare veri invarianti, favorire Aggregate piccoli, riferire gli altri Aggregate per identità e utilizzare la consistenza eventuale fuori dal confine quando il business la permette. Queste linee guida richiedono comunque giudizio sul caso reale.

5. Transactional Consistency e concorrenza

Una transizione valida in memoria non equivale automaticamente a una transizione valida nel database. Se due operatori caricano lo stesso WorkOrder, entrambi possono leggere uno stato compatibile con il proprio comando e tentare cambiamenti in conflitto. Il sistema deve proteggersi dai lost update.

Optimistic Concurrency Control

Una soluzione è associare una versione all'Aggregate. Il salvataggio richiede che la versione letta coincida ancora con quella presente nello storage. Quando l'aggiornamento fallisce per versione cambiata, occorre definire la risposta applicativa: segnalazione di conflitto, rilettura e nuova validazione o diversa strategia approvata.

```
UPDATE work_orders
SET status = 'COMPLETED',
    report_ref = :report_ref,
    minutes_worked = :minutes,
    version = version + 1
WHERE id = :work_order_id
  AND version = :expected_version;
```

Se la query modifica zero righe, il repository deve distinguere le possibili cause: Aggregate inesistente, cambio concorrente di versione, autorizzazione già scaduta o altro vincolo. Il retry cieco è rischioso, perché il dominio potrebbe essere cambiato e il comando potrebbe non essere più valido.

Altre opzioni sono i lock pessimisti e i vincoli atomici applicati direttamente dal database. La scelta dipende da frequenza dei conflitti, latency budget, isolamenti transazionali e rischio di business. L'uso di transazioni SQL non sostituisce il confine concettuale; è uno dei mezzi con cui lo rendiamo operativo.

Invariante cross-Aggregate

Supponiamo che un tecnico non possa essere assegnato a due lavori sovrapposti. Se il requisito è rigoroso, un controllo di disponibilità fatto su ciascun WorkOrder seguito da un evento asincrono non basta a evitare una doppia prenotazione concorrente. Potrebbe essere necessaria un'autorità di prenotazione che riservi atomicamente la capacità, un vincolo su una rappresentazione condivisa oppure una diversa modellazione.

Questa scelta non va automatizzata dal coding agent. Il domain expert deve chiarire se è ammissibile una sovrapposizione temporanea, quale sia la strategia di risoluzione, e come gestire modifiche o annullamenti.

6. Domain Services e Application Services

Il Domain Service rappresenta logica di dominio che non trova una collocazione naturale in una singola Entity o Value Object. Una politica di scelta del tecnico può dipendere da specializzazioni, distanze, priorità, capacità e preferenze contrattuali. Se tale valutazione è un concetto del business, può essere formalizzata in un Domain Service.

L'Application Service orchestra invece un caso d'uso: recupera dati, controlla input di applicazione, carica l'Aggregate, invoca le operazioni di dominio e coordina il commit. Non è il luogo in cui inventare o disperdere le policy di business.

```
// Pseudocodice indipendente dal framework
function completeWorkOrder(command): Result {
  transaction.begin();
  workOrder = repository.load(command.workOrderId);
  evidence = CompletionEvidence.create(
      command.reportRef, command.minutesWorked);
  workOrder.complete(evidence);          // regola dominio
  repository.save(workOrder);           // persistenza
```

```
    outbox.stage(workOrder.events());      // stessa transazione
    transaction.commit();
    return Success;
  }
```

Questo frammento semplifica la gestione dei fallimenti. In produzione si devono trattare rollback, conflitti, unit of work, propagazione degli eventi e deduplicazione. La responsabilità di una business policy resta comunque al modello e non al componente HTTP.

7. Domain Events: linguaggio dei fatti

Un Domain Event descrive un fatto significativo e già avvenuto. Un comando esprime una intenzione. `CompleteWorkOrder` può essere rifiutato; `WorkOrderCompleted` è il nome di un fatto conseguente a una transizione riuscita. Questo consente di esprimere chiaramente cause ed effetti nel dominio.

La Root può raccogliere eventi nel corso delle modifiche, ma la loro distribuzione non deve confondere la riuscita in memoria con il commit durevole. In presenza di più processi è importante distinguere Domain Event interno e Integration Event pubblico.

Un Integration Event è un contratto

Il messaggio pubblico dovrebbe contenere solo le informazioni approvate per i consumer. La sua forma può evolvere indipendentemente dalla rappresentazione interna dell'Aggregate, evitando di esporre dettagli che renderebbero i contesti semanticamente accoppiati.

```
type WorkOrderCompletedV1 = {
  eventId: string;
  workOrderId: string;
  occurredAt: string;
  schemaVersion: 1;
  reportReference: string;
};
```

La struttura è un modello candidato. Il significato di `reportReference`, la granularità degli identificativi, la retention e l'autorità dei consumer richiedono una decisione. Billing può ricevere la notizia del completamento, ma non deve interpretarla come un'autorizzazione automatica alla fatturazione.

8. Eventual Consistency: cosa garantiamo davvero?

Una consistenza eventuale consente che consumer e producer vedano temporaneamente situazioni differenti: Operations conferma un lavoro, mentre Billing deve ancora riceverne il fatto. Si tratta di un compromesso progettuale e non di una garanzia implicita che ogni cosa si sistemerà in modo autonomo.

La convergenza richiede condizioni operative: disponibilità della propagazione, retry affidabili, identificatori e versioni coerenti, gestione delle eccezioni, osservabilità e possibilità di riconciliare stati divergenti. Se un messaggio resta bloccato permanentemente, il sistema può restare incoerente per sempre nonostante l'etichetta "eventual consistency".

Decisione	Domanda da approvare
Ritardo	Quanto può rimanere non aggiornata la vista di Billing?
Failure	Quando un errore diventa incidente operativo o richiede intervento umano?
Retry	Quanti tentativi, con quali intervalli e condizioni di stop?
Reconciliazione	Come ricostruire lo stato dei messaggi mancanti o non elaborati?
Ordine	Come identificare e trattare eventi di versioni differenti?

9. Transactional Outbox: evitare il dual write

Il problema del dual write appare quando aggiorniamo il database e pubblichiamo un messaggio su un broker in due operazioni separate. Se il commit DB riesce ma la pubblicazione fallisce, altri contesti non ricevono il fatto. Se pubblichiamo prima e il commit poi fallisce, possono ricevere un fatto mai confermato.

Il pattern Transactional Outbox registra in una sola transazione locale lo stato del dominio e una riga Outbox. Un processo separato pubblica successivamente gli eventi pendenti. Questo riduce la finestra di perdita causata dal dual write, ma introduce una necessità di retry e monitoraggio del relay.

```
BEGIN TRANSACTION
  UPDATE work_orders ...;
  INSERT INTO outbox (
    event_id, aggregate_id, aggregate_version,
    event_type, payload, published_at
  ) VALUES (...);
COMMIT;

-- Relay separato:
-- recupera eventi pendenti, pubblica, marca il risultato
-- possibile duplicazione se crash dopo publish e prima del marker
```

L’Outbox non offre di per sé exactly-once delivery end-to-end. Anche con un relay ben progettato, la ripubblicazione può avvenire. È necessario proteggere il consumer dagli effetti duplicati.

10. Idempotent Consumer e ordinamento

Un consumer è idempotente quando ricevere lo stesso evento più volte produce lo stesso effetto di riceverlo una sola volta. La deduplicazione basata su `eventId` con un vincolo univoco è una delle tecniche concrete.

```
BEGIN TRANSACTION
  INSERT INTO processed_messages(subscriber_id, event_id)
  VALUES (:subscriber, :event_id);

  -- Se l'inserimento è già presente: non applicare
  -- nuovamente l'effetto di business.
  INSERT INTO billing_candidates(work_order_id, source_event_id)
```

```
    VALUES (:work_order_id, :event_id);
COMMIT;
```

L’inserimento dell’ID elaborato e l’effetto devono essere atomicamente coordinati. Un semplice `SELECT` seguito da `INSERT` senza vincolo unico potrebbe lasciare una race condition tra consumer concorrenti.

La deduplicazione non risolve l’ordine degli eventi. Se un evento relativo alla versione 6 arriva prima di uno della versione 5, è necessario decidere se bufferizzare, recuperare o ignorare eventi obsoleti in base al modello. La chiave di sequenza dovrebbe essere riferita a un flusso coerente, spesso quello di un Aggregate.

Test indispensabili

- Consegna dello stesso eventId due volte: un solo effetto.
- Due consumer simultanei sullo stesso eventId: assenza di duplicazione.
- Versione 6 prima della 5: nessuna regressione dello stato.
- Crash tra publish e aggiornamento marker: retry senza duplicazione del business.
- Messaggio invalido: rifiuto controllato, diagnosi e percorso operativo di recupero.

11. Saga e coordinamento di processi distribuiti

Una Saga coordina una sequenza di transazioni locali che costituiscono un processo di business. Può seguire un modello coreografico, nel quale i partecipanti reagiscono agli eventi, oppure un modello orchestrato, in cui un coordinatore esplicito decide i passi successivi.

Una compensazione non è un rollback tecnico del passato: è una nuova operazione di business, per esempio revocare una prenotazione. Potrebbe non essere disponibile per azioni già irreversibili, come l’invio di una comunicazione esterna o l’esecuzione materiale di un lavoro.

Un Process Manager con stato è una soluzione possibile per seguire processi lunghi. Non è obbligatorio per ogni interazione tra due contesti e non deve essere introdotto quando un semplice caso d’uso sincrono soddisfa tutte le esigenze.

Domanda critica

Se un passo successivo fallisce, quali effetti precedenti possono essere compensati? Quali devono essere rettificati? Quali richiedono un operatore umano? Queste risposte sono policy di business e rimangono UNKNOWN finché non vengono approvate.

12. Alternative e criterio di adozione

Approccio	È sensato quando...	Rischio da evitare
CRUD semplice	Pochi stati e regole lineari	Overengineering
Transaction Script	Caso d’uso limitato e chiaro	Regole duplicate in molti punti
Aggregate ricco	Invarianti e transizioni non banali	Aggregate eccessivamente grande
State Machine	Stati e transizioni sono il cuore del comportamento	Confondere statechart con tutto il dominio
Event-driven	Propagazione asincrona realmente utile	Recovery e semantica trascurati
Saga	Processo distribuito con failure parziali	Compensazioni improprie

Event Sourcing non è sinonimo di Domain Events. Può essere una scelta utile quando la cronologia degli eventi è fonte autorevole dello stato, ma comporta replay, versionamento degli eventi, migrazione dei dati e gestione delle proiezioni. Non deve essere introdotto soltanto perché abbiamo creato `WorkOrderCompleted`.

13. Specifiche agent-ready e verifica indipendente

Il modello diventa implementabile dagli agenti quando la specifica separa esplicitamente fatti, ipotesi, decisioni e questioni aperte. Una specifica deve descrivere confine, comportamenti, invarianti, eventi, concorrenza, errori, non-goals e criteri di verifica. Deve anche indicare chi può approvare modifiche semantiche.

```
# WORKORDER-AGGREGATE.md
Boundary: Operations
Aggregate Root: WorkOrder
R1: complete richiede stato ASSIGNED  [LAB DECIDED]
R2: rapporto e minuti positivi      [LAB DECIDED]
Concorrenza: version check          [DESIGN CANDIDATE]
Integration Event: WorkOrderCompletedV1 [PROPOSED]
Billing eligibility: UNKNOWN
SLA propagazione: UNKNOWN
Non-goal: decidere policy Billing

# WORKORDER-VERIFICATION.md
- transizioni valide e rifiutate
- invarianti sotto conflitti concorrenti
- outbox nello stesso commit locale
- duplicati e eventi fuori ordine
- contratto esterno e assenza di autorita Billing
```

Il coding agent può proporre soluzioni, ma la sua autorità resta subordinata a requisiti e decisioni approvate. Un secondo agente o una evaluation suite indipendente deve poter giudicare gli outcome usando criteri non generati automaticamente dalla stessa implementazione.

Le skill come `domain-modeling`, `to-spec`, `to-tickets`, implementazione/TDD e `code-review` facilitano il workflow; non sostituiscono l'esperto del dominio. Quando vengono introdotte in un progetto, occorre chiarire versione, fonte ufficiale e limiti, evitando di trasformare una convenzione di tooling in uno standard universale.

14. Failure mode e anti-pattern

Failure mode	Perché è pericoloso	Rimedio di progetto
Setters pubblici	Bypass degli invarianti	Comportamenti nella Root
Aggregate gigantesco	Conflitti e accoppiamento	Confini basati su true invariants
Validazioni solo nel controller	Altri entrypoint aggirano le regole	Validazioni di dominio
Publish prima del commit	Fatti mai confermati	Outbox / coordinamento corretto
Nessun consumer idempotente	Effetti di business duplicati	Idempotenza transazionale
Ignorare ordering	Stato che regredisce	Versioni per stream e recovery
Billing assume fatturabilità	Violazione dell'ownership	Policy autonoma del contesto
Test scritti dall'agent solo	Oracolo correlato all'implementazione	Criteri indipendenti

15. Laboratorio progressivo completo

Obiettivo: progettare una vertical slice concettuale del comando `CompleteWorkOrder` e delle sue conseguenze senza attribuire a Billing decisioni non approvate. Tempo suggerito: 12–15 minuti dopo la teoria, o come attività separata.

A. Modellazione

Definisci WorkOrder come Aggregate Root; CompletionEvidence come Value Object; TechnicianId come riferimento esterno. Scrivi una frase per ciascuna responsabilità e spiega perché non inserisci l'intero modello Technician all'interno dell'Aggregate.

B. Invarianti e transizioni

Given	When	Then atteso
OPEN	Complete valido	Rifiuto, nessun evento
ASSIGNED	Complete senza report	Rifiuto, nessun cambiamento
ASSIGNED	Complete valido	COMPLETED + evento raccolto
COMPLETED	Secondo Complete	Policy da chiarire, non assumere
Due writer v4	Commit concorrenti	No lost update silenzioso

C. Comunicazione con Billing

Proponi un contratto `WorkOrderCompletedV1` che pubblica un fatto approvato. In Billing l'evento può al massimo avviare una valutazione o creare una bozza/candidato, senza dichiarare autonomamente autorizzata una fattura. Definisci struttura Outbox, idempotenza consumer e verifica delle versioni.

D. Prove indipendenti

Prepara test con oracoli derivati da R1/R2 e da contratti approvati. Aggiungi almeno una prova concorrente e una prova di recupero dopo un evento duplicato. Per ogni `UNKNOWN`, crea una domanda da sottoporre ai domain expert; non produrre codice che finga una risposta.

Soluzione ragionata parziale

Una modellazione coerente per il laboratorio potrebbe impiegare WorkOrder come Root con `complete(evidence)`, VO validato alla creazione e repository con version check. Il caso d'uso salva Aggregate e Outbox nella stessa transazione. Il consumer Billing conserva gli eventId elaborati in modo atomico con i propri effetti. Questo è uno schema candidato, non una progettazione definitivamente approvata.

La parte più importante da discutere è la distinzione tra il fatto "lavoro completato" e la policy "servizio fatturabile". Il secondo concetto richiede autorità e requisiti del Bounded Context Billing. Se il laboratorio risolve questa incognita da solo, ha commesso un errore di Software Engineering anche se il codice funziona.

16. Esercizio di autovalutazione

Q1. È corretto creare un Aggregate per ogni tabella? **Risposta:** no, gli invarianti e la coerenza delle transizioni guidano il confine.

Q2. Un Value Object può contenere metodi? **Risposta:** sì, metodi coerenti con il valore, normalmente senza effetti collaterali e con risultati immutabili.

Q3. Un Domain Event è automaticamente un messaggio pubblicabile? **Risposta:** no; l'integration contract e il commit durevole sono questioni distinte.

Q4. Outbox garantisce un unico consumo? **Risposta:** no, il relay può ripubblicare; serve protezione dal duplicato.

Q5. Tutti gli invarianti possono essere eventualmente consistenti? **Risposta:** no, alcuni richiedono una garanzia atomica non differibile.

Q6. Che cosa deve fare l'agente quando incontra una business policy UNKNOWN? **Risposta:** registrarla, proporre alternative e chiedere autorità competente; non trasformarla in fatto.

17. Takeaway e collegamento alla Lezione 13

> **Entity = identità. Value Object = significato del valore. Aggregate = confine di consistenza. Root = autorità sulle transizioni. Eventi = fatti. Affidabilità = transazioni, recovery, idempotenza e verifica.**

L'AI non può compensare una modellazione ambigua semplicemente generando più codice o più test. Anzi, può moltiplicare rapidamente le violazioni quando i boundary non sono espliciti. La Lezione 13 userà questi confini per costruire Task Graphs, worktree, subagent e strategie di integrazione senza confondere il parallelismo tecnico con l'autorità sulle business policy.

Il laboratorio della Lezione 12 non è stato dichiarato completato: il corso avanza nella parte teorica, mantenendo aperto lo stato dell'esercitazione.

Bibliografia e note delle fonti

Le fonti seguenti sono state controllate durante la preparazione della lezione. I riferimenti fondamentali restano validi indipendentemente dal framework e non devono essere interpretati come prescrizioni univoche per ogni progetto.

Vaughn Vernon, Effective Aggregate Design (Part I–III)
https://kalele.io/effective-aggregate-design/

Martin Fowler, DDD Aggregate
https://martinfowler.com/bliki/DDD_Aggregate.html

Microsoft Learn, Design a microservice domain model
https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/microservice-domain-model

Microsoft Learn, Domain events: Design and implementation
https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/domain-events-design-implementation

Chris Richardson, Transactional Outbox
https://microservices.io/patterns/data/transactional-outbox

Chris Richardson, Idempotent Consumer
https://microservices.io/patterns/communication-style/idempotent-consumer.html

Chris Richardson, Saga Pattern
https://microservices.io/patterns/data/saga.html